Tenant Metering, Throttling, and Chargeback Design & Architecture Document

Version: 1.1 | Date: January 12, 2026 | Owner: DevSecOps

Requirement References

- PHO-41: As a platform owner, I want to track hardware resource consumption per tenant so that I can monitor usage, enforce limits, and plan capacity. IN PROGRESS
- PHO-42: As a platform owner, I want each application isolated so that no single app can consume all hardware resources and impact others. DONE

1. Executive Summary

JIFFY.ai is building a multi-tenant low-code/no-code platform on AWS called **Phoenix**. The platform includes about 20 microservices running on AWS Lambda and ECS and uses managed services such as Cognito, API Gateway, RDS (PostgreSQL), S3, ElastiCache, OpenSearch, CloudWatch, Amazon MQ, Bedrock, Secrets Manager, and WAF. Given multi-tenancy and usage-based commercial models, the platform requires consistent tenant-level metering, soft-to-hard throttling, and invoice-ready internal chargeback per customer (tenant) and per application (tenant+app).

This document proposes a unified “Tenant Usage Platform” consisting of four building blocks:

(1) Tenant Context propagation, (2) Standardized Usage Events, (3) Quota/Throttle Engine (soft then hard), and (4) Billing Ledger + Cost Allocator to generate monthly chargeback invoices (USD).

2. Background, Scope, and Goals

2.1 Scope

In scope:

- Tenant identity resolution via subdomain and Cognito claims; consistent propagation to all services.
- Metering for API, workflows, documents, AI, storage, search, MQ, and compute proxies.
- Quota and throttling enforcement at the edge (API Gateway) and internally (Temporal and service layer).

- AI Gateway for token estimation and AI budget enforcement with a simplified token rate card.
- Invoice-ready internal monthly chargeback generation per tenant and per app, including shared-infrastructure allocations.

2.2 Goals

- Correctness: prevent double-billing, maintain audit-grade traceability, and ensure deterministic invoice regeneration.
- Scalability: support up to 1000 tenants while only allocating real-time quota state for active tenant+app keys.
- Fairness: protect platform from noisy neighbors via per-tenant+app rate limiting and concurrency controls.
- Predictability: simplified rate card for AI tokens; clear soft vs hard behavior and operator controls.
- Extensibility: support future “tenant cell” isolation for high-volume/regulatory tenants without changing metering.

3. Assumptions and Constraints

- UI is served through CloudFront; API traffic enters primarily through API Gateway (public).
- Tenant identifier is available from both subdomain (tenant.jiffy.ai) and Cognito JWT custom claims.
- Temporal is used for orchestration; there is one Temporal cluster per environment (dev/stage/prod).
- Quota enforcement is required at tenant+app granularity (a tenant can have multiple apps).
- S3 storage is per-tenant bucket (improves attribution accuracy).
- Billing is internal chargeback (not a legal tax invoice), monthly, in USD.
- Some tenants may require dedicated resources in the future; platform APIs remain common.

4. High-level Architecture Overview

The design uses an event-based metering pipeline and a Redis-backed Quota Service for real-time enforcement. Billing relies on an immutable Billing Ledger and monthly aggregation, augmented by CUR-based shared-cost allocation.

4.1 Reference Architecture (logical)

```
CloudFront (UI)
  |
```

```
  v
API Gateway + WAF
  |
  v
Lambda Authorizer / JWT Auth
```

- Validate JWT
- Resolve tenant via subdomain + JWT claim
- Enforce tenant(subdomain) == tenant(JWT)
- Call Quota Service (soft/hard decision)

```
  |
  v
Platform Services (Lambda/ECS)
```

- Shared tenant-context + metering library
- Emit Usage Events

```
  |
  +--> Temporal (Workflows/Activities)
  |     - Interceptors enforce quotas + concurrency
  |     - Emit workflow/doc usage events
  |
  +--> AI Gateway (Bedrock/OpenAI/Anthropic)
```

- Token estimation/reporting
- Enforce token budgets
- Emit AI usage events

```
Usage Events -> EventBridge/Kinesis -> Firehose -> S3 (Raw Usage Lake)
                                  |
                                  v
                        Usage Curator (Dedupe/Validate)
                                  |
                                  v
                          Billing Ledger (DynamoDB)
                                  |
                                  v
```

Monthly Invoice Generator (USD)
+ Shared Cost Allocator (CUR)

5. Tenant Identity and Context Propagation

5.1 Tenant Resolution

Tenant is resolved at ingress using both the request subdomain (tenant.jiffy.ai) and Cognito JWT custom claims (e.g., custom:tenant_id, custom:plan). The system must reject mismatches to prevent tenant spoofing via host headers.

5.2 Tenant Context Contract

After authentication/authorization, inject and propagate the following headers/fields end-to-end:

- x-tenant-id
- x-app-id
- x-tenant-plan
- x-request-id
- x-trace-id

5.3 Shared Library

A shared library (for Lambda and ECS services) standardizes: (a) tenant-context extraction, (b) propagation to outbound HTTP and MQ messages, and (c) emission of standardized usage events. This eliminates inconsistent metering across services.

6. Metering and Usage Events

6.1 Billable Units

Domain	Metered Units
API	requests, response_bytes, latency_ms
Workflows (Temporal)	workflow_runs, workflow_steps
Documents	pages_processed, ocr_seconds
AI (Bedrock/OpenAI/Anthropic)	ai_input_tokens, ai_output_tokens, ai_invocations, model
Storage (S3)	s3_bytes_stored, s3_put/get requests (proxy units)

Search (OpenSearch)	opensearch_query_count, index_bytes
Messaging (MQ)	messages_published, messages_consumed
Compute proxies (optional)	lambda_ms, ecs_cpu_ms (estimated)

6.2 Usage Event Schema (minimum)

Each billable action emits a Usage Event with the following minimum fields:

- event_time, env
- tenant_id, app_id, plan
- service, operation, product
- unit, quantity, dimension (e.g., model, workflow type, region)
- request_id, trace_id
- idempotency_key (stable across retries to prevent double-billing)

6.3 Transport and Storage

Recommended pipeline: EventBridge (or Kinesis for higher throughput) → Firehose → S3 (raw immutable usage lake).

A Usage Curator validates and deduplicates events into a DynamoDB Billing Ledger for audit-grade invoices.

7. Quotas and Throttling (Soft then Hard)

7.1 Enforcement Layers

- Edge (API Gateway): fast gate for request rate limiting and burst control.
- Workflow/Compute (Temporal + services): enforce concurrency and protect expensive operations (workflows, doc processing, AI).

7.2 Quota Service

A Quota Service provides real-time rate limiting and concurrency enforcement. Redis (ElastiCache) stores windowed counters and semaphores for active tenant+app keys only (TTL-based). Monthly budgets are evaluated using a cached remaining-budget object (TTL 60-300 seconds) to avoid synchronous DynamoDB calls on every request at 1M-tenant scale.

7.3 Soft vs Hard Behavior

Soft actions (between soft and hard thresholds):

- API: add small controlled delay for non-critical endpoints, or return 202 for async-able operations.
- Temporal: reduce per-tenant+app concurrency and allow queueing (Temporal naturally queues).
- AI: lower max tokens per request or switch to a smaller model where allowed.
- Docs: queue processing and return clear user feedback ('processing delayed due to quota')

Hard actions (beyond hard threshold):

- API: return 429 with Retry-After and clear error payload.
- Temporal: reject new workflow starts with QuotaExceeded.
- AI Gateway: deny invocations once token budgets are exhausted.
- Docs: stop new document processing activities.

8. Redis Key Model (Scale-safe)

8.1 Windowed Rate Limit Keys

Use TTL + time-windowed keys so only active tenant+app keys consume memory.

Purpose	Key Pattern	Notes
Per-API op RPM	rl:{env}:{tenant}:{app}:{op}:{epoch_min}	integer count; TTL 2-3 minutes
Workflow start RPM	rlwf:{env}:{tenant}:{app}:{wf}:{epoch_min}	integer count; TTL 2-3 minutes
AI tokens/min	rltk:{env}:{tenant}:{app}:{model}:{epoch_min}	token sum; TTL 2-3 minutes

8.2 Concurrency Semaphores

Purpose	Key Pattern	Notes
Workflow concurrency	sem:wf:{env}:{tenant}:{app}	atomic Lua check+increment; TTL safety

AI concurrency	sem:ai:{env}:{tenant}:{app}	atomic Lua check+increment; TTL safety

All increments must be atomic (Lua script) to avoid race conditions at high concurrency. Keys must include environment to prevent cross-env interference.

9. Temporal Enforcement (Tenant+App)

9.1 Workflow Start Interceptor

On workflow start: resolve tenant_id/app_id/plan and call QuotaService.AcquireWorkflowSlot(...).

On completion: always call ReleaseWorkflowSlot(...) in a finally handler to prevent leaked slots.

9.2 Activity Interceptor

Before expensive activities (doc processing, AI invocation, high-fanout steps), acquire an activity slot or budget. Emit usage events for steps, pages, and external calls. AI calls must be routed via AI Gateway.

10. AI Gateway and Token Metering

To support invoice-ready AI chargeback, introduce an AI Gateway microservice as the single choke point for Bedrock/OpenAI/Anthropic calls. The gateway standardizes metadata, estimates token counts when provider usage is unavailable, enforces token budgets (tokens/minute and tokens/month), and emits ai.tokens and ai.invocation usage events.

10.1 Token Measurement

- If provider response includes token usage: record tokens_reported.
- Otherwise estimate tokens using tokenizer libraries (e.g., tiktoken for OpenAI; approximations for Anthropic). Record tokens_estimated.
- Billing uses tokens_reported when available; otherwise uses tokens_estimated.

10.2 Simplified Token Rate Card

Use a simplified, predictable price per 1M tokens (combined input+output) for internal chargeback.

11. Cost Attribution and Allocation

11.1 Direct vs Shared Cost

Direct costs are preferred where resources are tenant-dedicated and tagged. Shared costs require allocation using measurable drivers derived from metering and service statistics.

11.2 RDS Allocation (DB-per-tenant on shared instance)

AWS billing does not provide per-database RDS cost. Allocate RDS monthly cost using driver-based weights per tenant database. Collect metrics hourly/daily via a DB Metrics Collector job:

- db_size_bytes via pg_database_size(dbname)
- activity/load via pg_stat_database (xact count, blocks read/hit, temp bytes)
- optional query cost via pg_stat_statements (total_time, calls)

Example allocation formula:
TenantShare = 0.5*(CPU_time proxy) + 0.3*(DB_size) + 0.2*(IO_blocks)

11.3 S3 Attribution

S3 uses per-tenant buckets, enabling strong attribution. Track bytes stored (via app + S3 Inventory reconciliation) and request counts (proxy via app operations).

11.4 OpenSearch Allocation

Prefer per-tenant index or index pattern. Track index size by tenant and query volume by tenant (instrumented in app/proxy). Allocate domain cost as: 0.6*index_size + 0.4*query_volume.

11.5 ElastiCache and MQ Allocation

ElastiCache (Redis) is shared; allocate using per-tenant cache operation counts and/or keyspace bytes (tenant-prefixed keys).

MQ costs can be allocated by per-tenant message volume when tenant is a routing dimension and consumers emit usage events.

12. Billing System (Usage to Invoice)

12.1 Components

- Billing Ledger (DynamoDB): immutable, deduped billable events for audit-grade invoicing.
- Rate Card Service: price per unit by tier (requests, runs, steps, pages, tokens, storage, search, messages).

- Monthly Invoice Generator: aggregates ledger, applies rate card, adds shared allocations, emits monthly chargeback invoice JSON.
- CUR Ingest + Allocator: CUR → Athena queries monthly AWS costs; allocator distributes shared pools using drivers.

12.2 Invoice Output

Chargeback invoices must include: usage summary, line items (unit price x quantity), shared allocations, and totals (USD).

12.3 Sample Invoice JSON


```
{
  "tenant_id": "t-123",
  "month": "2026-01",
  "currency": "USD",
  "apps": [
    {
      "app_id": "app-9",
      "line_items": [
        {"unit":"api_requests","quantity":3200000,"rate_per_unit":0.0000002,"amount":0.64},
        {"unit":"workflow_runs","quantity":42000,"rate_per_unit":0.0005,"amount":21.00},
        {"unit":"doc_pages","quantity":180000,"rate_per_unit":0.0002,"amount":36.00},
        {"unit":"ai_tokens","quantity":74000000,"rate_per_unit":0.0015,"amount":111.00}
      ],
      "shared_infra_allocations": [
        {"pool":"rds_shared","driver":"db_size+activity","amount":18.20},
        {"pool":"opensearch_shared","driver":"index_size+query_volume","amount":9.10}
      ],
      "total": 195.94
    }
  ],
  "tenant_total": 195.94,
  "notes": ["Internal chargeback estimate based on metered usage + allocation pools."]
}
```


13. Sample Tier Model (Tenant + App)

The following baseline tiers can be tuned based on observed usage patterns.

Metric	Free (Trial)	Pro	Enterprise
API requests / month / app	100k	5M	25M (configurable)
Workflow runs / month / app	1,000	50k	250k
Steps / month / app	50k	5M	25M
Doc pages / month / app	5,000	500k	5M
AI tokens / month / app	2M	100M	1B
Workflow concurrency / app	2	20	100
AI concurrency / app	1	10	50
Soft threshold	90%	90%	90%
Hard threshold	110% (stop)	120% (bill)	configurable

14. Dedicated Tenant Resources (Tenant Cells)

For tenants requiring regulatory or performance isolation, introduce “tenant cells” (dedicated resources) while keeping a common API surface. Routing can direct a tenant to shared backend or to a dedicated cell (dedicated ECS services, OpenSearch, and/or RDS). Metering remains identical; cost becomes more direct via resource tagging.

15. Rollout Plan

Phase 1: Tenant+App context enforcement and standardized usage event library across Lambda/ECS/Temporal.

Phase 2: Quota Service + Redis gates; edge enforcement in API Gateway; Temporal interceptors for concurrency and workflow rate.

Phase 3: AI Gateway rollout with token estimation and token budgets; emit ai.tokens usage events.

Phase 4: Billing Ledger (DynamoDB), Rate Card Service, and Monthly Invoice Generator (USD) with internal dashboards.

Phase 5: CUR-based shared cost allocator + allocation pools (RDS/OpenSearch/Redis/MQ); tenant cell path for top tenants.

16. Key Design Decision: Active Tenant+App Model

To support infinite tenants safely, real-time quota state (Redis counters, budget cache) is maintained only for active tenant+app keys using TTL and windowed key patterns. The authoritative billing record is append-only (S3 raw lake + DynamoDB ledger).

17. Risks and Mitigations

Risk	Mitigation
Double-billing due to retries	Require idempotency_key on all usage events; dedupe in Usage Curator before ledger write.
Redis hot keys for very high-volume tenants/apps	Shard keys by operation and window; consider 10s windows; tenant cell for extreme tenants.
Token estimation variance	Prefer provider-reported usage; store estimator version in dimension; monitor variance.
Quota-service availability	Deploy HA; fail-open only for non-critical endpoints with strict internal caps; alerting and runbooks.
Shared cost allocation disputes	Use transparent formulas and publish drivers on invoices; refine weights based on observed behavior.